Contents

Acknowledgements **iv**

List of figures **v**

List of tables **vii**

Abbreviations **viii**

List of Symbols **x**

Abstract **xi**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Proposed Approach 2

1.4 Objectives and Contributions 2

1.4.1 Objectives 2

1.4.2 Contributions 2

1.4.3 Scope 3

1.4.4 Organisation 4

2 Literature Review and Research Gap **5**

2.1 Scope of the Review 5

2.2 Digital Twins for Network Slicing 5

2.3 Self-Supervised Learning for Wireless Data 5

2.4 Traffic Forecasting for Slicing 6

2.5 Reinforcement Learning for Slicing 6

2.6 Data Used in the Reviewed Work 7

2.7 Research Gap 7

3 Datasets and Data Preparation **9**

3.1 Introduction 9

3.2 Datasets Used 9

3.3 Is there a Unified Public Dataset? . . . 10
3.4 Dataset Limitations . . . 10

4 System Design and the Unified Slicing Environment 12
4.1 Overview . . . 12
4.2 System Architecture . . . 12
4.3 Problem Formulation . . . 13
4.4 The Unified Slicing Environment . . . 13
4.5 Calibration and Realism Check . . . 14
4.6 Reference Policies . . . 14
4.7 Implementation and the Role of the Digital Twin . . . 15
4.8 Summary . . . 16

5 Self-Supervised Representation Learning 17
5.1 Motivation and Objective . . . 17
5.2 Pretext Tasks . . . 17
5.2.1 First implementation: masked reconstruction . . . 17
5.2.2 Re-evaluation: two pretext tasks on cleaned data . . . 18
5.3 Cleaning the Data . . . 18
5.4 Label-Efficiency Study . . . 18
5.5 SSL on the Milan Forecast . . . 20
5.6 Interpretation . . . 20

6 Temporal GNN Traffic Forecasting 21
6.1 Forecasting Objective . . . 21
6.2 Graph Construction Across Datasets . . . 21
6.3 GraphSAGE–GRU Architecture . . . 22
6.4 First Experiments on the Milan Block . . . 22
6.5 City-Wide Generalisation Failure . . . 22
6.6 The Corrected City Forecaster . . . 22
6.7 Limitations of the Forecasting Study . . . 25

7 MARL for Slice Allocation 26
7.1 Allocation Problem . . . 26
7.2 Scalable MARL Design . . . 26
7.3 Evaluation Protocol . . . 26
7.4 MARL versus a Single Central Agent . . . 27
7.5 Comparison with Properly Tuned Rules . . . 27
7.5.1 What the learned controller does . . . 30
7.6 Training Variants . . . 30
7.7 On-Policy Baselines . . . 30
7.8 Robustness to Traffic Shifts . . . 31

7.9 Earlier B5G MADDPG Development . . . 32
7.10 Chapter Conclusion . . . 32

8 Integrated Evaluation and the Digital Twin 34
8.1 Introduction . . . 34
8.2 Value of the Forecast . . . 34
8.3 The Forecast and the Learned Controller . . . 35
8.4 Controllers Across the Whole City . . . 36
8.5 The Digital Twin Dataset . . . 37
8.6 What-if Scenarios . . . 38
8.7 The Digital Twin as an Approval Gate . . . 39
8.8 Checking the Surrogate Against a Packet-Level Simulator . . . 40
8.9 The Role of the Digital Twin in This Work . . . 41
8.10 Summary . . . 42

9 Prototype: Slice Twin Monitor 43
9.1 Introduction . . . 43
9.2 Purpose and Design . . . 43
9.3 Views and Controllers . . . 44
9.4 Technical Notes . . . 44
9.5 Contribution to the Project . . . 45

10 Conclusion and Scope for further Research 47
10.1 Summary . . . 47
10.2 Contributions . . . 48
10.3 Limitations . . . 48
10.4 Future Work . . . 49
10.5 Concluding Remarks . . . 50

A Dataset Columns and Experimental Settings 51
A.1 Columns of the Digital Twin Dataset . . . 51
A.2 Experimental Settings . . . 52

References 53

List of Publications Based on the Research Work 57

Planned Publications 58

Acknowledgements

We express our sincere gratitude to our thesis advisor, Dr. Sundaresan S, for guidance and constructive feedback throughout this project. His questions about why a multi-agent controller is needed, and how a forecast should be shown to help, shaped the experiments in this thesis. We thank the faculty and review panel of the Department of Artificial Intelligence for helping us sharpen the problem statement and connect the implementation to a defensible evaluation.

We thank Amrita Vishwa Vidyapeetham, Coimbatore Campus, for the academic environment in which this work was undertaken, and Dr. K.P. Soman, Professor and Dean of the Department of Artificial Intelligence, for the facilities provided. We also acknowledge the creators of the public datasets and the maintainers of the open-source software on which this work builds: the Telecom Italia Big Data Challenge, 5G-NIDD, NeversNet5G and the B5G slicing dataset, and OMNeT++, INET, Simu5G, PyTorch and PyTorch Geometric. Their openly available data and documentation made it possible to concentrate on the unified environment, fair evaluation and the Digital Twin.

We thank our teammates, friends and families for their support. The responsibility for the implementation choices, the interpretation of results and the remaining limitations rests with the authors.

KEERTHIVASAN S V
KRISH S
SRIRANJANA C
THRISHALA S N

List of Figures

4.1 Architecture of the proposed system 13
4.2 Reference Simu5G NR run (one gNodeB, one UE, 5 s, downlink VoIP) 16

5.1 Macro-F1 against the number of labelled flows for raw features and pretrained encoders 19

6.1 Real traffic on the Milan grid 23
6.2 Training and validation error of the first Milan forecaster 23
6.3 City-wide forecaster on the held-out week: five days of training against 39 days . 24

7.1 Reward against network size: shared MARL, the single agent and the reference rules 28
7.2 Long-protocol MARL against the tuned rules on the held-out week 29

8.1 Value of prediction for the expert rule at four network sizes 35
8.2 Learned controller with and without the forecast 36
8.3 Delay against load: Simu5G and the surrogate of the environment (normalised) . 40

9.1 Data flow of the Slice Twin monitor 45
9.2 City view at 17:30: cells coloured by real traffic (top, dark theme) and by SLA misses under the Reactive controller (bottom, light theme). 46

List of Tables

1.1 Stages of the proposed pipeline 2

2.1 Summary of the reviewed papers by pillar 6

3.1 Datasets used in the project 9

4.1 Components of the unified environment 14
4.2 SLA targets of the three slices 14

5.1 Macro-F1 on 81,731 unseen 5G-NIDD flows, by number of labelled flows (mean of five label draws up to 5,000 labels, two above) 19
5.2 City forecaster with and without a pretrained window embedding, improvement over persistence on the held-out week (eMBB / URLLC / mMTC) 20

6.1 First Milan block experiment (one day, 400 cells, one seed) 22
6.2 City-wide evaluation of the forecaster trained on the 100 central cells only (eMBB mean squared error, $Mbps^2$, two held-out days) 23
6.3 Improvement over persistence on the held-out week, 16–22 December 2013 (eMBB / URLLC / mMTC) 24
6.4 Mean squared error of simple baselines and the TGNN on all 10,000 cells ($Mbps^2$), held-out week 25

7.1 Held-out week reward at a training budget of 1,000 episodes (mean over seeds, number of seeds in brackets) 27
7.2 Policy size as the network grows 27
7.3 Long protocol (4,000 episodes, checkpoint chosen on validation days, actors averaged in logit space) against the tuned expert rules, held-out week (difference in reward, 95% CI, days better out of 7) 29
7.4 What the learned controller does differently (25 cells, held-out week; SLA rates, fraction of demand served, and mean capacity share per slice) 30
7.5 Training variants at 25 cells, held-out week reward 31
7.6 On-policy baselines at 25 cells (8,000 episodes, checkpoint chosen on validation days), held-out week 31
7.7 Controllers trained on ordinary traffic, run on shifted traffic (25 cells, held-out week, DDPG-MARL averaged over three 2,000-episode seeds) 32

7.8 MARL trained on a mixture of traffic scenarios (two seeds, actors averaged), against an expert rule re-tuned on each scenario's own training days 32

8.1 Reward of the expert rule with different knowledge of the next ten minutes, held-out week (default targets) 34
8.2 MARL with and without forecast input (25 cells, three seeds, mean ± sample standard deviation) 35
8.3 Controllers on all 10,000 cells, Wednesday 6 November 2013 (reward of the whole city, the dense block and the outer city) 36
8.4 Size of the Digital Twin dataset 37
8.5 Scenarios of the core year 38
8.6 Effect of each injected event (core year, events scenario) 38
8.7 Night-time sleep across the city (01:00 to 05:00, 62 days) 39
8.8 Effect of the approval gate on the held-out week (25 cells, mean over the three MARL seeds) 39
8.9 Simu5G load sweep (10 UEs, one gNodeB, downlink capacity 56.1 Mbps, 1000-byte messages, 6 s, mean of three repetitions) 40
8.10 Controllers on the Simu5G-shaped latency curve (25 cells, held-out week) 41

9.1 Controllers on the 100 central cells, Wednesday 6 November 2013 44

Abbreviations

6G	Sixth-generation mobile communication network
AI	Artificial intelligence
DDPG	Deep deterministic policy gradient
DL	Downlink
DT	Digital Twin
eMBB	Enhanced mobile broadband
GNN	Graph neural network
gNB	Next-generation Node B; a 5G/6G base station
GAE	Generalised advantage estimation
GRU	Gated recurrent unit
IPPO	Independent proximal policy optimisation
LLM	Large language model
MAE	Mean absolute error
MADDPG	Multi-agent deep deterministic policy gradient
MAPE	Mean absolute percentage error
MAPPO	Multi-agent proximal policy optimisation
MARL	Multi-agent reinforcement learning
MCP	Model Context Protocol
MLP	Multilayer perceptron
mMTC	Massive machine-type communication
MSE	Mean squared error
NDT	Network Digital Twin
NR	New Radio
O-RAN	Open Radio Access Network
PPO	Proximal policy optimisation
POMDP	Partially observable Markov decision process
PRB	Physical resource block
QoS	Quality of service
RAN	Radio access network
RL	Reinforcement learning
RMSE	Root mean squared error
SCARF	Self-supervised contrastive learning using random feature corruption
SLA	Service level agreement
SINR	Signal-to-interference-plus-noise ratio
SSL	Self-supervised learning
TGNN	Temporal graph neural network

UE	User equipment
URLLC	Ultra-reliable low-latency communication
VNF	Virtual network function

List of Symbols

i	Index of a network cell or cell-level agent
t	Time-step or traffic interval index
$\mathcal{V}$	Set of cells (nodes) in the network graph
$\mathcal{E}$	Set of edges representing connections or neighbourhood relations between cells
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Network graph, consisting of cells and their relationships
s_t^i	State or observation available to the agent controlling cell i at time t
a_t^i	Allocation action selected by the agent for cell i at time t
R_t^i	Reward received by the agent controlling cell i at time t
S_t^i	Weighted service-level-agreement (SLA) score for cell i at time t
F_t^i	Fraction of demand served by cell i at time t
H_t^i	Reconfiguration churn for cell i at time t
$D_{t,k}^i$	Demand for slice k in cell i at time t
$\widehat{D}_{t+1,k}^i$	Forecast demand for slice k in cell i for the next interval
C_t^i	Usable capacity of cell i at time t
$x_{t,k}^i$	Fraction of cell i's capacity allocated to slice k at time t
$x_{t,\text{idle}}^i$	Fraction of cell i's capacity left idle at time t
k	Index identifying a network slice, such as eMBB, URLLC or mMTC
L	Network latency or packet/frame delay, depending on the reported experiment
J	Jitter, representing variation in packet/frame delay
ρ	Reliability or probability of successful service, when used in the SLA definition
$\mathbf{z}_t^i$	Learned feature representation (embedding) for cell i at time t
$\widehat{\mathbf{y}}_t$	Model-predicted value or traffic demand; the exact target depends on the experiment

Abstract

Network slicing lets enhanced mobile broadband (eMBB), ultra-reliable low-latency communication (URLLC) and massive machine-type communication (mMTC) services share one infrastructure. Most slicing controllers only react once demand has already changed. In this thesis we ask two questions: does it help to predict traffic before bandwidth is allocated, and is a decentralised multi-agent controller a better design than one central controller?

We built a pipeline of self-supervised learning (SSL), a temporal graph neural network (TGNN) forecaster and multi-agent reinforcement learning (MARL), and tested it in a Digital Twin. No public dataset combines topology, per-slice traffic, quality of service and allocation actions, so we made a unified slicing environment from real Telecom Italia Milan traffic (62 days, 10,000 cells), with latency calibrated on a public 6G RAN telemetry dataset. All results are measured on a held-out week, and the controllers were chosen on separate validation days.

A MARL controller with shared parameters keeps one actor of 19,332 parameters from 9 to 100 cells. A single central agent grows to 604,048 parameters and its reward falls from about 0.43 to 0.27. We also compared with rule-based controllers whose targets were tuned on the training days. With a long training budget and averaged seeds, the off-policy learned controller beats them on all seven test days at every size we tried, by 0.023 to 0.026 in reward. It only matches a richer rule that is tuned for each time of day and cell class. An on-policy (PPO-family) learner does beat that rule on every test day, by 0.009 in reward at 25 cells.

A TGNN trained on 39 days beats persistence on all 10,000 cells (its mean squared error is lower by 18%, 33% and 31% for eMBB, URLLC and mMTC). Giving its forecast to the rule-based controller raises the reward by about 0.03, which closes about one fifth of the gap to perfect knowledge of the next interval. We have not shown that the forecast helps the learned controller. Self-supervised pretraining, tested on de-duplicated data with different numbers of labels, gave no gain over strong raw-feature baselines, and we report this as a negative result.

The Digital Twin works as an approval gate that protects the reward from poor proposals. A packet-level Simu5G sweep agrees with our calibrated QoS model on the order of delays but not on the shape of the delay curve. We also release a Digital Twin dataset of 393.6 million rows with six what-if scenarios, and an interactive monitor, Slice Twin, which is published on the web.

Keywords: network slicing, 6G, Digital Twin, self-supervised learning, temporal graph neural network, multi-agent reinforcement learning, traffic forecasting.

Chapter 1

Introduction

1.1 Background and Motivation

Sixth-generation networks will have to carry very different services over the same radios. Streaming video and extended reality want high throughput (enhanced mobile broadband, eMBB). Factory control and connected vehicles want a delay of a few milliseconds and almost no loss (ultra-reliable low-latency communication, URLLC). Sensor networks want to connect huge numbers of cheap devices that each send very little (massive machine-type communication, mMTC).

Network slicing is the usual answer. The operator cuts the radio and core resources into logical slices and promises each slice a service level agreement (SLA). The difficult part is deciding how big each slice should be, because demand keeps moving and is different in every cell. A fixed split wastes capacity when a slice is quiet and breaks the SLA when it is busy. Learning-based controllers do better, but most of the ones we found in the literature only look at what the network is doing right now, so they react after the demand has already moved.

1.2 Problem Statement

We ask two questions in this thesis.

1. Can a slicing controller that is told the demand of the next interval allocate bandwidth better than one that sees only the current demand?

2. Is a decentralised multi-agent controller a better solution to this task than a single central agent?

We also want to try such controllers in a Digital Twin (DT) before anyone would use them on a real network.

1.3 Proposed Approach

Our pipeline has four stages (Table 1.1). A self-supervised learning (SSL) encoder is trained on unlabelled telemetry and gives a compact description of each observation. A temporal graph neural network (TGNN) predicts the demand of every slice in the next interval; it mixes information from neighbouring cells with GraphSAGE [1] and follows each cell through time with a recurrent update. A multi-agent reinforcement learning (MARL) controller, with one agent per cell, then splits the capacity of its cell between eMBB, URLLC, mMTC and an idle share. The Digital Twin is a virtual copy of the network where these allocations are trained, tested and, if possible, approved before they are applied.

Table 1.1: Stages of the proposed pipeline

Stage	**Role**	**Output**
Self-supervised learning	Learns representations of telemetry from unlabelled data by masking or corrupting features and recovering them	Embedding per node and time step
Temporal GNN	Forecasts next-interval demand per cell and slice from the cell graph and recent history	Predicted demand
Multi-agent RL	Each cell agent decides how to split its bandwidth between slices	Slice shares per cell
Digital Twin	Virtual network that trains, tests and checks allocations and supports what-if scenarios	Evaluation, approval, scenario data

1.4 Objectives and Contributions

1.4.1 Objectives

1. Survey the literature on Digital Twins, SSL, graph-based forecasting and MARL for network slicing and identify the research gap.
2. Collect and prepare public data for each stage of the pipeline.
3. Implement each module and evaluate it against strong, simple baselines on real data.
4. Provide one environment in which prediction and allocation are tested together on the same traffic.
5. Build a working prototype.

1.4.2 Contributions

Each contribution below is supported by experiments in the later chapters, and the limits of each are stated there.

1. **A unified slicing environment.** No public dataset combines topology, per-slice traffic, quality of service (QoS) and allocation actions. We built one from real Telecom Italia Milan traffic, with latency calibrated on a public 6G radio access network (RAN) telemetry dataset, and we mark every component as real, calibrated or modelled.

2. **A fair evaluation of MARL.** A parameter-shared controller keeps a fixed-size actor from 9 to 100 cells while a central agent does not scale. Against expert rules tuned on the training days, it wins on all seven test days at every size (by 0.023 to 0.026 in reward), and an on-policy PPO-family learner also beats a rule tuned per time of day and cell class (by 0.009 at 25 cells, on every test day), whereas the off-policy learner only matches that rule.

3. **A measurement of what prediction is worth.** Giving the expert rule the TGNN forecast raises its reward by about 0.03 on every test day, about one fifth of the gap to perfect knowledge. The benefit to the learned controller is not established.

4. **A city-scale forecasting finding.** A forecaster trained only on the dense centre was 10.8 times worse than persistence in the outer city. Per-cell scaling, random spatial patches and more training days fixed this, and the retrained forecaster beats persistence in all 10,000 cells (18%, 33% and 31% lower error for the three slices).

5. **A clean negative result for SSL.** After removing duplicate flows and leaked columns, pretraining gave no gain over strong raw-feature baselines at any label budget, on 5G-NIDD and on the Milan forecaster.

6. **A Digital Twin that checks decisions.** An approval gate in the environment protects the reward from poor proposals, and a Simu5G load sweep tests the QoS model against a packet-level simulator.

7. **A dataset and a prototype.** A 393.6-million-row, 122-column dataset with six what-if scenarios, and the Slice Twin monitor, which is published on the web.

1.4.3 Scope

The thesis does not cover the following.

- A packet-level simulator inside the training loop. The QoS comes from a calibrated surrogate, and the telemetry used to calibrate it is itself simulated. Our own 19-gNodeB scenario is not connected to Simu5G; we only ran a single-cell load sweep.

- A statistically wide evaluation. Results use one held-out week (seven days), at most three seeds per learned controller (one at 100 cells), and one city.

- Deployment on a live or emulated network. The prototype replays recorded days.

1.4.4 Organisation

Chapter 2 reviews the literature and the research gap, and Chapter 3 describes the datasets and how we prepared them. Chapter 4 presents the system design, the problem formulation and the unified environment. Chapters 5, 6 and 7 cover self-supervised representation learning, temporal GNN forecasting and MARL for slice allocation. Chapter 8 gives the integrated evaluation, the Digital Twin dataset, the approval gate and the Simu5G check. Chapter 9 describes the Slice Twin prototype, and Chapter 10 ends with the limitations and future work.

Chapter 2

Literature Review and Research Gap

2.1 Scope of the Review

This chapter summarizes the literature on Digital Twins, self-supervised learning (SSL), graph-based forecasting, and reinforcement learning (RL) for slicing that is relevant to the four proposed building blocks of the pipeline. The papers studied span the years 2020–2026 from IEEE venues and some were arXiv or TechRxiv preprints. We picked up the method, data or simulator, results and research gap for each. The papers are summarized pillar wise in Table 2.1.

2.2 Digital Twins for Network Slicing

In [2] a three-layer native Network Digital Twin (NDT) architecture, with a four-stage lifecycle (preparation, creation, runtime, feedback) was proposed. In an operator-inspired signalling-storm scenario, it achieved approximately 3% MAPE when considering the registration arrival rate and approximately 25% for the successful registration count, but it only allows for simulation and verification and does not include any GNN, SSL or RL. Graph-based twins predict network KPIs: one predicts end-to-end slice latency with less than 5% error on NSFNET, GEANT2 and a synthetic topology with 50 nodes [3]; a virtualisation-aware twin adds graph attention to improve performance over RouteNet-Fermi [4]; a decentralised GNN twin addresses end-to-end slicing [5]. These twins are based on the static graphs and supervised learning. Twins with control agents are still reactive: using a clean telemetry in the same way, a twin-synchronised DDPG agent reduced latency by approximately 25% compared to static allocation [6]; an NDT agent on the Milan data (of about 10,000 cells) achieved 25.5% less load and 30.4% less energy savings with a rolling-average predictor [7].

2.3 Self-Supervised Learning for Wireless Data

A survey on SSL for wireless networks is presented that investigates the generative and contrastive and predictive paradigm and the integration with Digital Twins, TGNNs and reinforce-

ment learning is still not widely explored [8]. Net2Net learns node embeddings from predicting the position and achieves better results than baselines on eight graph datasets, with only static and non-telecom graphs [9]. For network traffic, two-stage SSL was able to achieve 97.64% accuracy on network traffic classification for encrypted traffic in SDN, which is approximately 2% higher than a supervised model [10]; autoencoder with tabular contrastive learning and Confident Learning achieved improved label noise reduction on three traffic datasets [11]. The use of SSL is therefore one of classification or non-telecom graphs, and is uncommon for slicing telemetry.

2.4 Traffic Forecasting for Slicing

The Holt-Winters forecasting has been integrated with adaptive VNF scaling for slicing, however, only in simulated topology and traffic [12] and DeepCog forecasts capacity for slice provisioning [13]. Compared to other graph-based methods, SSGNN achieves best improvements of up to 3.0% in MAE, 5.2% in RMSE and 3.8% in MAPE across six road-traffic datasets [14]. It forecasts road traffic and the forecast is not linked to any allocation decision.

2.5 Reinforcement Learning for Slicing

Other representative multi-agent approaches to slicing include SF-DTMC with an adaptive retention framework that reduced reconfiguration time and SLA violations in NS-3 but was limited to local cell clusters and reactive observations [15]; MADDPG for vehicular slicing, achieving 96.5% QoS satisfaction with coordination delays ranging from 120–180 ms which violate URLLC budgets [16]; and an improved MADDPG with rank voting for multi-objective allocation, with no Pareto guarantee [17]. Other work include: GAN-DDPG with a semantic-aware reward (22% and 20% higher URLLC and eMBB spectral efficiency, respectively, and 31% lower packet loss) [18] ; DRL for end-to-end slicing on a real testbed (where safety DRL cut SLA violations from up to 15% to about 1%, and learning on a real network is slow and potentially unsafe) [19] ; slice mobility by optimisation and game theory [20] ; online learning for AI-service slicing [21] ; an agentic LLM control plane with closed-loop SLA monitoring [22] ; and a conceptual semantic "Reasoning Plane" [23]. This thesis aims to place the semantic slicing framework as future work.

Table 2.1: Summary of the reviewed papers by pillar

Ref.	**Method**	**Data used**	**Gap**
Digital Twin			
[2]	Three-layer native NDT, gray-box modelling	Signalling-storm scenario (private)	Single use case; no GNN, SSL or RL
[3]	GraphSAGE twin for end-to-end latency	NSFNET, GEANT2, synthetic	Static graph; supervised
[4]	GraphSAGE + graph attention, virtualisation-aware	Docker UPF, extended OMNeT++	Static graph; supervised

Ref.	Method	Data used	Gap
[5]	Decentralised GNN twin	OMNeT++ (120,000 train, 3,120 validation)	Real-time data, diverse traffic, isolation, SLA unresolved
[6]	Twin synchronisation + DDPG	Own Python 5G simulator	Clean telemetry assumed; reactive
[7]	Rolling average, Isolation Forest, MLP	Milan (public)	Simple predictor; one dataset
Self-supervised learning			
[8]	Survey of SSL paradigms	None (survey)	Integration with twins, TGNN, RL unexplored
[9]	Position-prediction SSL	Eight non-telecom graph datasets	Static graphs
[10]	Pre-training and fine-tuning	Mininet, D-ITG traffic	Classification only
[11]	Autoencoder, contrastive learning, Confident Learning	ISCX VPN-nonVPN, UCDavis–QUIC, own flows	Limited datasets; generalisation unvalidated
Forecasting			
[12]	Holt-Winters, VNF scaling, DPSO and annealing placement	Simulated 5G topology and traffic	No real-world validation; scalability
[14]	Masked SSL, spatio-temporal GNN, self-distillation	METR-LA, PEMS-BAY, PEMS03/04/07/08	Road traffic; not used for allocation
Reinforcement learning and slicing management			
[15]	SF-DTMC in RAN-POSG	NS-3 simulator	Local clusters; reactive
[16]	POMDP, MADDPG	Mininet, Ryu	Simulation only; delays exceed URLLC budget
[17]	MADDPG with rank voting	Simulated RAN, three slice types	Simulation; no Pareto guarantee
[18]	GAN-DDPG, semantic reward	Custom 6G simulator	Domain-specific semantic states
[19]	Safety, distributed and imitation DRL	OpenAirInterface/FlexRAN testbed	Reactive; real-network learning unsafe
[20]	Optimisation, game theory	Discrete-time simulation	Optimisation-based; no SSL, TGNN or MARL
[21]	Online learning (EXP3)	Mobile-health dataset	No forecasting, no MARL
[22]	LLM agents, MCP, SLA monitoring	Slice-specification, manifest and intent data; Open5GS and Ericsson RAN testbed	Large-scale, multi-domain, adversarial robustness
[23]	Semantic Reasoning Plane	None (conceptual)	Added latency not quantified

2.6 Data Used in the Reviewed Work

The remaining twelve papers rely on data from a simulator, emulator or testbed, while only one paper relies on public cellular traffic, which is also used by us [7]. None use a public data set that includes topology, per-slice traffic, QoS and allocation decisions, so generating that data is the common practice, which is accommodated by the unified environment of Chapter efchap.4.

2.7 Research Gap

Individual components of our pipeline are discussed in the literature: reinforcement learning-based twins, slicing using MARL, and forecasting using GNN. Most controllers use the current state; forecasts are seldom related to allocations; graph models are based on static graphs or

road traffic; and multi-agent work is tested on small clusters and own simulators. We did not identify a paper that integrates the combination of SSL, temporal graph forecasting, a Digital Twin and MARL for slicing, while other papers we reviewed included an overview of the different building blocks but not how the forecasts are used to allocate. This thesis is a test of forecasting and multi-agent allocation jointly on real city-scale traffic.

Chapter 3

Datasets and Data Preparation

3.1 Introduction

Different kinds of data are required at each stage of the pipeline and no one source contains all of these. This chapter outlines the datasets gathered for the project, the reasons that none of these constitute a single dataset and the limitations associated with each. Table 3.1 lists the datasets that were collected and used.

3.2 Datasets Used

Table 3.1 gives the size, contents, role and preprocessing of each dataset.

Table 3.1: Datasets used in the project

Dataset	**Size**	**Contents**	**Role**	**Preprocessing**
Telecom Italia Milan	62 days, 10,000 cells, 10-minute intervals	SMS, call and internet activity on a 100×100 grid of 235 m cells	Real traffic for SSL, TGNN, the unified environment and the Digital Twin dataset	Internet, calls and SMS mapped to eMBB, URLLC and mMTC demand; per-cell 95th-percentile scaling for the city-wide forecaster
5G-NIDD	1,215,890 flows (408,655 unique)	Labelled benign and attack flows from a 5G network	Training and evaluation of the SSL encoder	Exact duplicates and identifier columns removed, log scaling, one-hot encoding, stratified 80/20 split

Dataset	Size	Contents	Role	Preprocessing
NeversNet5G	716 UE files, about 27 GB, 19 gNodeBs	Vehicular 5G NR telemetry produced with Simu5G and SUMO	UE to gNodeB topology (695 nodes) and telemetry replay	Graph built by nearest-gNodeB association
B5G network slicing	7,984 samples	Transport topologies with eMBB, URLLC and mMTC flows	First MARL environment	Train, validation and test split
6G RAN telemetry	30,000 windows	PRB load, latency percentiles, SLA flag and reward	Calibration of latency against load	Latency and jitter curves fitted against load
DeepSlice, Secure5G, Kaggle slicing sets, Botez 6G slicing	Small	Slice labels and QoS budgets	Slice-type benchmarks	—

3.3 Is there a Unified Public Dataset?

A dataset that links all the stages of the pipeline would have to hold, in one record, the topology of the network, the traffic of each cell over time, the demand of each slice, QoS measurements, the SLA outcome, and the allocation action taken with its reward. We searched the datasets we hold and the web, and none of them contains all of these.

The closest is the B5G slicing dataset, which covers only the transport network and has no record of allocation actions. Published O-RAN slicing traces come from single-cell testbeds, and we could not confirm that they are released. The literature audit of Chapter 2 agrees: of the sixteen papers reviewed, eight use their own data sets, and none releases a public dataset with topology, per-slice traffic, QoS and allocation decisions together.

On the advice of our guide we therefore built a unified dataset synthetically, based on real data wherever real data exist. The environment is described in Chapter 4 and the data it generates in Chapter 8.

3.4 Dataset Limitations

Each source limits what the experiments can show.

- **Telecom Italia Milan** has no network-control information: no PRB allocation, slice configuration, latency, QoS or SLA decisions, and no MARL actions. It supplies the real traffic-demand layer only, and the demand of each slice is assumed from it.
- **B5G** models the transport network and not the RAN. The samples used do not include latency or loss, so the reward of the first MARL environment is nearly trivial, and no Digital Twin loop could approve actions on it.

- **6G RAN telemetry** calibrates latency against load, but the telemetry is itself simulated. The calibration therefore ties the environment to a model and not to measurements from an operating network.
- **NeversNet5G** was produced with Simu5G and SUMO and not on a real deployment.
- **5G-NIDD** is a labelled intrusion-detection dataset. It can evaluate the SSL encoder, but it holds no slicing information. About two thirds of its flows are exact duplicates (807,235 of 1,215,890), which would leak between a random training and test split, so only the unique flows are used.

Chapter 4

System Design and the Unified Slicing Environment

4.1 Overview

The four phases of the proposed pipeline (Table 1.1) can only be evaluated collectively, if the forecast, allocation and the resulting service quality are known for the same cell within the same interval. This chapter outlines this: the formulation of the slicing task, unified environment for evaluation of forecast and allocations in the testbed, calibration process for the testbed's quality-of-service (QoS) model, and reference policies used to compare learned controllers.

A dataset that links these phases would have to hold, in one record, the network topology, the traffic of each cell over time, the demand of each slice, QoS measures, the SLA outcome, and the allocation action with its reward. We reviewed the datasets we hold and searched for others, and none has all of these. The closest are the B5G slicing dataset, which covers only the transport network and has no allocation actions, and published O-RAN slicing traces, which come from single-cell testbeds and whose release we could not verify. We therefore built the unified environment synthetically, based on real data wherever real data exist.

4.2 System Architecture

The architecture of the proposed system consists of eight stages, as shown in Figure 4.1. Network data are gathered from multiple sources (stage 1). Self-supervised learning learns representations from unlabelled network data (stage 2), while the temporal graph neural network makes predictions for future traffic and network states using temporal graphs (stage 3). Multiple MARL agents make joint decisions (stage 4). These decisions are executed in the Digital Twin, which is a virtual copy of the network (stage 5), and resources are assigned to the network slices according to the predictions (stage 6). The network state in the Digital Twin is updated (stage 7), and its performance is evaluated to calculate the reward to improve the policies of the agents (stage 8). Stages 5 to 8 are implemented in the unified environment presented in Section 4.4.

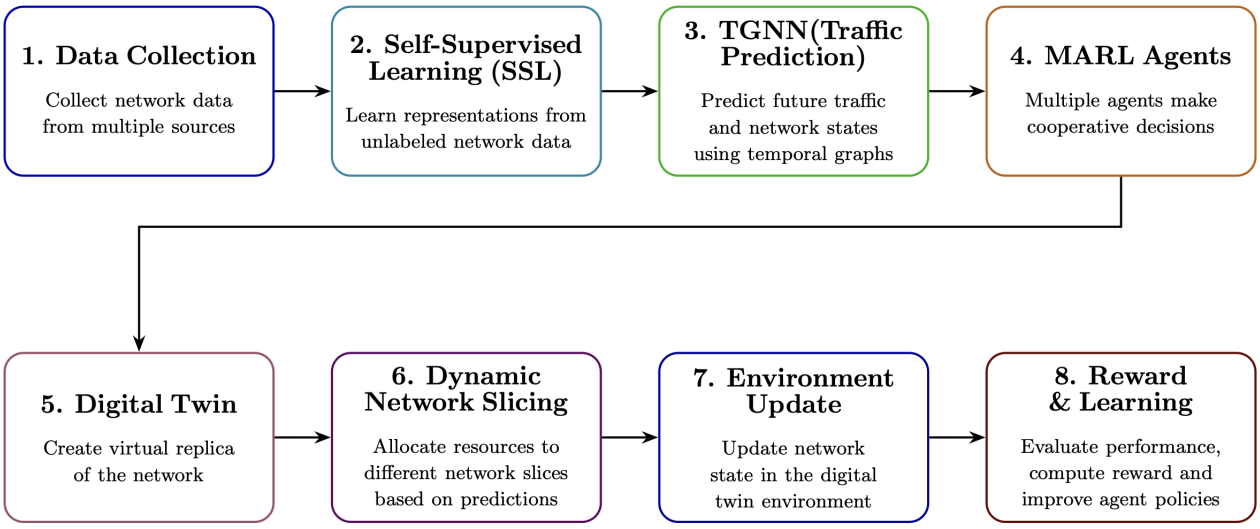


Figure 4.1: Architecture of the proposed system

4.3 Problem Formulation

Each cell is controlled by one agent. The agent splits the capacity of its cell into four portions for each of eMBB, URLLC, mMTC and idle and applies the selected portions to the next interval, and so on. Each agent in MARL takes an action only based on 17 local features for their own cell. The load of neighbouring cells diminishes the capacity of a cell (Section 4.4), which means that the decisions of the agents are coupled, and the task is a decentralised multi-agent control problem.

The reward of the agent of cell i in interval t is

$$R_t^i = 0.7\, S_t^i + 0.3\, F_t^i - 0.1\, H_t^i, \tag{4.1}$$

where S_t^i is a weighted SLA score, F_t^i is a fraction of demand that is satisfied and H_t^i is a reconfiguration churn. The weights are design choices. The reward therefore favours meeting the SLA targets of the three slices first, then serving demand, and penalises changes of the allocation. During training, each agent's reward is mixed with those of its neighbours (the team reward).

4.4 The Unified Slicing Environment

The environment brings together real traffic on a real spatial layout, per-slice demand, an allocation action, and the resulting QoS, SLA outcome and reward for each cell and each 10-minute interval. It is implemented in the `src/unified` folder of the project repository. Table 4.1 shows which components are real, which are calibrated and which are modelling assumptions.

The traffic and the grid layout are real. The mapping from activity type to slice and the scaling of each slice to the 100 Mbps cell capacity are assumptions, because the Milan data

Table 4.1: Components of the unified environment

Component	Status	Source or rule
Traffic per cell and interval	Real	Telecom Italia Milan
Topology	Real	Milan grid layout with 4-connected neighbours
Slice demand	Modelled	Internet activity mapped to eMBB, calls to URLLC and SMS to mMTC, scaled so that the 95th percentile of each slice is 80%, 10% and 5% of a 100 Mbps cell
Latency and jitter against load	Calibrated	Curves fitted to the 6G RAN telemetry dataset (30,000 windows)
Inter-cell interference	Modelled	Usable capacity = 100 Mbps / (1 + 0.3 × mean neighbour load)
Loss and SLA	Rules	Targets given in Table 4.2
Action	Agent output	Shares of each cell's capacity for eMBB, URLLC, mMTC and idle, applied to the next interval
Reward	Design choice	Equation (4.1)

Table 4.2: SLA targets of the three slices

Slice	Latency	Jitter	Reliability
eMBB	50 ms	20 ms	0.99
URLLC	5 ms	1 ms	0.99999
mMTC	100 ms	50 ms	0.99

has no slice information. Latency and jitter as functions of load are the calibrated part and are described next.

4.5 Calibration and Realism Check

To check realism, the eMBB SLA rate of the model was compared with the SLA label of the reference telemetry across fifteen load bins. The deterministic latency model fitted poorly, with a mean difference of 0.086. Adding lognormal spread to the latency, with a single parameter $\sigma = 0.21$, reduced the difference to 0.021. This is not an independent validation, because σ was fitted to the same curve.

Two further points limit how far the QoS model can be trusted. It is a calibrated surrogate and not a packet-level simulator, and the reference telemetry used to calibrate it is itself simulated. In addition, the reinforcement learning experiments use the deterministic version of the latency model, without the lognormal spread. Section 8.8 compares the model with a packet-level simulator.

4.6 Reference Policies

In addition to the learned controllers, there are three reference policies that are also tested in the environment:

- **Equal split**, splits the cell's capacity equally.
- **Proportional allocation** means allocating the capacity according to the demand.
- **Expert policy**, which allocates sufficient capacity to URLLC and mMTC to maintain their utilisation within the SLA-safe region and the remaining capacity to eMBB.

The expert policy is run with three inputs: the current demand, the TGNN forecast of the next interval, and the true demand of the next interval, which is an upper bound. Comparing these three versions measures the value of knowing the next interval without involving a learning method. The rule has two utilisation targets for URLLC and mMTC. Besides the default targets, two tuned versions are used (Chapter 7), so that learned controllers are compared with rules that were optimised on the same training days.

4.7 Implementation and the Role of the Digital Twin

The environment and the controllers are implemented in Python with PyTorch, and 25 automated tests pass.

In parallel, a packet-level simulation tool chain was set up: OMNeT++ 6.4.0, INET 4.7.0 and Simu5G 1.7.0 under WSL2. In the reference NR run, a single-cell stand-alone scenario with one gNodeB and one UE, 5 simulated seconds of downlink VoIP were simulated. 28,092 events were processed, 93 of 93 packets were delivered, the mean VoIP frame delay was 4.82 ms and the MOS was 4.41 (Figure 4.2). The same tool chain is used for the load sweep of Section 8.8.

The Digital Twin is the unified environment together with the data generated from it (Chapter 8). In the early B5G orchestrator an approve-or-reject gate was prototyped, but it could never approve an action because B5G has no telemetry on which to check an SLA. The unified environment has a working gate: it predicts the outcome of a proposed allocation with the environment's QoS model and the TGNN forecast and applies a safe fallback when the proposal looks worse (Section 8.7).

Chapter 5

Self-Supervised Representation Learning

5.1 Motivation and Objective

Self-supervised learning (SSL) lets a model learn useful descriptions of network telemetry without a label for every observation. We hoped these descriptions could feed the later stages, a classifier or the forecaster, and that they would matter most when only a few labels are available. This chapter checks that idea. It describes the encoder, the data problems that first made our results look worse than they were, and a study that asks the question SSL is meant to answer: does pretraining help when only a few flows are labelled?

5.2 Pretext Tasks

5.2.1 First implementation: masked reconstruction

Our first encoder was trained on 5G-NIDD. Eleven context features (protocol, TCP state, TTL and hop attributes, among others) were treated as categories with an 8-dimensional embedding each. Fourteen volume and rate features (packet and byte counts, mean packet sizes, load and rate) were hidden with probability 0.30 and had to be rebuilt from the visible ones. The encoder had two linear layers of 128 and 64 units and gave a 64-dimensional embedding, and a decoder with one hidden layer of 128 units rebuilt the hidden values. The loss is the mean squared error over the hidden positions only,

$$\mathcal{L}_{\text{mask}} = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} (\hat{x}_j - x_j)^2, \tag{5.1}$$

where $\mathcal{M}$ is the set of masked positions, x_j the original value and $\hat{x}_j$ the reconstruction. Training used Adam with a learning rate of 10^{-3}, a batch size of 512 and 10 epochs, on all 1.2 million flows.

5.2.2 Re-evaluation: two pretext tasks on cleaned data

For the final study we redid the pretraining with two standard tabular pretext tasks. Both use the same encoder (83 inputs, two hidden layers of 256 units, a 64-dimensional output), so the comparison between them is fair.

- **Masked-feature modelling.** About 30% of the entries of each flow are swapped with the values of the same feature from other flows. The encoder has to recover the original values and also say which entries were swapped [24].
- **Contrastive learning (SCARF).** Each flow is paired with a copy in which 60% of the features are replaced by values taken from other random flows. A contrastive loss over the batch (temperature 0.5, batch size 512) pulls the two views of the same flow together [25].

Both were trained without any label on the 80% pool of flows kept for training (326,924 flows).

5.3 Cleaning the Data

The first evaluation reported a macro-F1 of 0.708. Two data problems explain why that number was so low, and fixing them changed the baseline far more than any SSL variant.

- **Duplicate flows.** Of the 1,215,890 flows in 5G-NIDD, 807,235 (66%) are exact copies of other flows. With a random split, identical flows land in both the training and the test set. Once they are removed, 408,655 unique flows remain, in nine classes (benign and eight attack types).
- **Identifier columns.** The row index, the sequence number and the offset tell where a flow sits in the capture, and that gives away its label, so we dropped them. The other numeric features are heavy-tailed counts, so we took their logarithm, and we one-hot encoded the categorical ones. That leaves 83 inputs.

A stratified 80/20 split leaves 326,924 flows for pretraining and 81,731 for testing. We had also found earlier that the source file is stored in blocks of identical labels. A split by position therefore put 99.88% of one class into the test set and gave an accuracy of about 1%. The stratified split removes that problem as well.

5.4 Label-Efficiency Study

Pretraining should help when labels are scarce, so we trained classifiers on 50, 200, 1,000, 5,000 and 20,000 labelled flows, and on all labels. Each budget up to 5,000 was repeated with five random draws of the labelled flows, and larger budgets with two. The classifiers were logistic regression, a small multilayer perceptron and LightGBM [26] on the raw features; logistic regression on the frozen embedding; the pretrained encoder fine-tuned with a classification

LightGBM is the weakest method at 50 labels, because boosted trees need more data. That is why we also compare against the neural and linear raw-feature models.

5.5 SSL on the Milan Forecast

In our first forecasting experiment (Chapter 6) a TGNN fed with 64-dimensional embeddings of Milan traffic scored the same validation error as one fed with the raw features (0.000616 against 0.000612). That was one day and one seed, so we repeated the test on the city forecaster. The embedding is now 16-dimensional and describes the input window of one cell: the last six intervals of demand, the same time on the previous day, the time of day and a weekend flag. The encoder is pretrained without labels. It hides 30% of the entries of a window and rebuilds them, using windows from the training days only. It is then frozen, and its output goes to the output layer of the forecaster. Table 5.2 gives the outcome. Both variants use the same data, split, scaling and number of training steps; the only difference is the embedding. The pretrained embedding does not help: with it the eMBB improvement is about one percentage point lower (15% against 16%), and URLLC and mMTC are within half a point of the raw forecaster. Each variant was trained twice with different seeds, and the run-to-run difference is of the same size as the gap, so we conclude that the embedding adds nothing measurable.

Table 5.2: City forecaster with and without a pretrained window embedding, improvement over persistence on the held-out week (eMBB / URLLC / mMTC)

Forecaster input	**Runs**	**eMBB**	**URLLC**	**mMTC**
Raw inputs (the city forecaster as it is)	2	+16%	+32%	+31%
Raw inputs + frozen 16-dimensional SSL embedding	2	+15%	+32%	+31%

5.6 Interpretation

The finding is the same on both tasks: the raw features already hold what the downstream models need, and masked or contrastive pretraining adds very little. A likely reason is that these are low-dimensional, hand-made features and not raw packets or long sequences. Pretraining is known to pay off on high-dimensional raw inputs, where no one has engineered features. Other work on tabular data reports the same thing, namely that the advantage of pretraining often disappears against well-tuned baselines [27]. We therefore report SSL as a negative result of this project. What this chapter contributes is the cleaned evaluation: de-duplicated data, no leaked columns, a design that varies the number of labels, and strong raw-feature baselines. The SSL stage stays in the pipeline of Chapter 4 because the literature recommends it, but we do not claim that it improves the forecasts or the allocation.

Chapter 6

Temporal GNN Traffic Forecasting

6.1 Forecasting Objective

The forecasting stage predicts the demand of every cell for the next ten minutes, separately for the three slices, using the recent history of the cell and of its neighbours. In the unified environment (Chapter 4) the prediction is a vector of three numbers in Mbps per cell. The earlier B5G and NeversNet5G experiments used a similar network to predict throughput, latency and load. A softplus kept the first two non-negative, and a sigmoid kept the load between 0 and 1.

6.2 Graph Construction Across Datasets

The implementation supports three graph sources.

Milan grid. The traffic lies on a regular 100×100 grid. Each cell is a node joined to its four neighbours, so the graph is the real geographic neighbourhood. The first experiments used a central 20×20 block of 400 cells. The final forecaster covers all 10,000 cells.

NeversNet5G. Here the graph snapshots come from event-driven UE telemetry and keep the identities of the UEs and gNodeBs and how they relate over time. The 64-dimensional SSL embeddings we exported are aligned with the graph timestamps. When a gNodeB has no embedding of its own, we average the embeddings of the UEs connected to it.

B5G topology. The B5G pipeline uses the graph, the routing matrix and the slice configuration that come with the dataset. It replays recorded samples instead of running an online simulator, and the recorded delta value serves as the reward.

6.3 GraphSAGE–GRU Architecture

At every time step two GraphSAGE layers combine each node's features with the mean of its neighbours' features,

$$h_v^{(l+1)} = \sigma\big(W^{(l)}\left[h_v^{(l)} \,\|\, \mathrm{mean}_{u\in\mathcal{N}(v)}\, h_u^{(l)}\right]\big), \tag{6.1}$$

where $h_v^{(l)}$ is the representation of node v at layer l, $\mathcal{N}(v)$ its neighbours, $W^{(l)}$ a weight matrix and σ an activation [1]. The sequence of spatial representations over the observation window is fed to a GRU cell, and the final hidden state is used to predict the next interval. The temporal part is a plain `GRUCell` over the output of PyTorch Geometric's `SAGEConv`, not the graph-convolutional GRU of the original design, because `torch_sparse` did not install on the Windows development machine.

6.4 First Experiments on the Milan Block

The first experiment used 400 cells over one day of 144 ten-minute steps, with a window of five steps (111 training and 28 validation windows) and 25 epochs of Adam at a learning rate of 0.0005. We compared the TGNN with persistence and with the training mean. The TGNN

Table 6.1: First Milan block experiment (one day, 400 cells, one seed)

Model	**Validation MSE**
GraphSAGE–GRU TGNN	0.0006160
Persistence	0.0007196
Training-mean predictor	0.0063946

was about 14% better than persistence. With only one day and one seed this shows that the model can learn the short-term pattern, not that it generalises. Feeding the five raw features in place of the SSL embeddings gave 0.0006122, so the embedding added nothing (Chapter 5).

6.5 City-Wide Generalisation Failure

We applied the forecaster trained on the 100 central cells to all 10,000 cells of the city, and it failed on the quiet ones. It had only seen the dense centre, so it had learned that traffic is heavy, and it over-predicted the suburbs by about 6 Mbps on average. On the 9,600 outer cells its eMBB error was 10.8 times that of persistence.

This is a level-bias and spatial-coverage problem: a model that has never seen a quiet cell cannot predict one.

6.6 The Corrected City Forecaster

The architecture was kept and the training was changed in four ways.

is worse than persistence on eMBB in this week. So the amount of training data, and not the architecture, was the main limit.

Table 6.4 compares the model with two seasonal baselines. At a horizon of ten minutes persistence is very hard to beat. The value at the same time on the previous day gives 8.1 times the eMBB error of persistence, and the average for that time of day gives 13.6 times. Persistence is therefore the right baseline to beat.

Table 6.4: Mean squared error of simple baselines and the TGNN on all 10,000 cells ($Mbps^2$), held-out week

Forecaster	**eMBB**	**URLLC**	**mMTC**
Repeat the last value (persistence)	1.449	0.127	0.061
Same time on the previous day	11.799	0.666	0.193
Average for that time of day	19.736	0.522	0.152
TGNN, trained on 39 days	1.190	0.084	0.042

6.7 Limitations of the Forecasting Study

We trained each forecaster once, so we have no estimate of the variation between seeds. We also did not separate the contributions of per-cell scaling, patch sampling, the extra inputs and the graph aggregation, and we have no graph-free baseline (a GRU on each cell alone) or published spatio-temporal baselines such as STGCN. These tests would show which part of the model does the work, and they are listed as future work in Chapter 10. The value of the forecast to the controllers is measured in Chapter 8.

Chapter 7

MARL for Slice Allocation

7.1 Allocation Problem

The last learned component shares the capacity of each cell between the eMBB, URLLC and mMTC slices. Every cell has its own agent. The agent sees 17 local quantities: the demand, utilisation and QoS of each slice, its previous allocation, the time of day, and the forecast when one is given. It outputs four non-negative numbers, and a softmax turns them into the shares of eMBB, URLLC, mMTC and idle capacity. The environment of Chapter 4 then works out the QoS, the SLA outcomes and the reward. The cells affect each other only through interference, since the usable capacity of a cell drops when its neighbours are loaded.

7.2 Scalable MARL Design

All cells share one actor, a network with 19,332 parameters that maps the observation of a cell to its allocation. The actor never looks at other cells, so the same copy works at any network size. Training uses a centralised critic that looks at the neighbourhood: it takes the observation and action of a cell together with the mean observation and mean action of its grid neighbours. The reward of each agent also contains a share of its neighbours' rewards, which pushes the agents to coordinate locally.

The learning algorithm is an actor–critic in the style of MADDPG [28]. It uses a replay buffer of 50,000 transitions, a discount factor of 0.9, soft target updates with $\tau = 0.01$, learning rates of 3×10^{-4} for the actor and 10^{-3} for the critic, and exploration noise added in logit space. The earlier B5G trainer used the observed reward as a one-step target, because its offline samples are single decisions and not interactions with a simulator.

7.3 Evaluation Protocol

Every result in this chapter uses the same protocol. Agents train on days 1 to 40 of the recorded data. Days 41 to 44 are only used to pick a checkpoint or tune a rule. The test is the held-out week of 16 to 22 December 2013, with one reward per day, and each table entry is the

mean of those seven days. We compare two controllers day by day and quote a 95% interval on the mean difference, using a t value for six degrees of freedom. Consecutive days are not independent, so the intervals only describe how much the days of this one week vary; they say nothing about another week or another city. Learned policies are also averaged over several seeds, and every table gives the number of seeds.

7.4 MARL versus a Single Central Agent

The first question is whether splitting the problem into local agents matters. A single central agent sees the observations of all cells and outputs all the allocations at once. We trained it with two learning rates, on the same data, reward and number of episodes. Table 7.1 shows the result at a budget of 1,000 episodes.

Table 7.1: Held-out week reward at a training budget of 1,000 episodes (mean over seeds, number of seeds in brackets)

Cells	**Shared MARL**	**Single agent (lr 10^{-4} / 3×10^{-4})**	**Equal split**	**Expert, current**	**Expert, forecast**	**Expert, forecast, tuned**	**Expert, true next**
9	0.926 ± 0.005 (2)	0.433 / 0.442	0.785	0.794	0.825	0.909	0.945
25	0.833 ± 0.086 (2)	0.337 / 0.329	0.750	0.775	0.804	0.877	0.920
100	0.857 (1)	0.284 / 0.267	0.735	0.765	0.794	0.865	0.908

The shared actor keeps both its size and its reward as the network grows. The central actor, in contrast, grows from 114,468 to 604,048 parameters (Table 7.2) and its reward falls from about 0.43 to 0.27–0.28. This supports the split into local agents. The central agent has to learn a very large action space from one learning step per episode step, and in our runs its reward went down during training instead of up. So the result tells us that this central baseline works badly, not that every centralised design would. Sharing parameters is the natural way to keep the problem manageable, and that is the point of the comparison.

Table 7.2: Policy size as the network grows

Cells	**Shared actor parameters**	**Single-agent actor parameters**	**Central input / output size**
9	19,332	114,468	153 / 36
25	19,332	200,548	425 / 100
100	19,332	604,048	1700 / 400

7.5 Comparison with Properly Tuned Rules

A learned controller should also be compared with the strongest simple one, not only with an equal split. The expert rule gives URLLC and mMTC enough capacity to hold their utilisation

7.5.1 What the learned controller does

Table 7.4 compares the allocations. The learned controller gives URLLC and mMTC about twice the capacity that the untuned rule gives them (shares of 0.23 and 0.09 against 0.11 and 0.04), so their SLAs are almost always met. It accepts a slightly lower share of eMBB demand served, and overall the reward is higher. This is a sensible policy and not an accident of the numbers, and the tuned rule ends up in the same place.

Table 7.4: What the learned controller does differently (25 cells, held-out week; SLA rates, fraction of demand served, and mean capacity share per slice)

Controller	**SLA met (eMBB / URLLC / mMTC)**	**Demand served**	**Capacity share (eMBB / URLLC / mMTC)**	**Reward**
Shared MARL (2,000 episodes, seed 0)	0.68 / 0.99 / 1.00	0.92	0.68 / 0.23 / 0.09	0.904
Expert rule, forecast, untuned	0.72 / 0.80 / 0.61	0.95	0.85 / 0.11 / 0.04	0.804
Expert rule, true next demand	0.72 / 1.00 / 1.00	0.95	0.85 / 0.11 / 0.04	0.920
Equal split	0.18 / 0.99 / 1.00	0.77	0.33 / 0.33 / 0.33	0.750

7.6 Training Variants

We tried several ways to improve the learner at 25 cells (Table 7.5): explicit communication between neighbouring actors, an expert-residual policy in which the actor outputs a correction to the allocation of the rule, a loss regularised towards the expert in the style of TD3+BC [29], and richer observations. None of them is clearly better than the plain shared actor trained with the long protocol and averaged. The on-policy learners of Section 7.7 are the exception. An actor with richer observations does not beat the rich tuned rule (a difference of -0.007 ± 0.009, better on 2 of 7 days). The variants with one seed are only hints and should not be read as a ranking.

7.7 On-Policy Baselines

To compare with the off-policy design we also implemented independent PPO (IPPO) and a MAPPO-style variant [30–32]. The policy is a Gaussian over the four logits. Advantages come from generalised advantage estimation [33] with $\lambda = 0.95$ and $\gamma = 0.9$ on the team reward, the clipping parameter is 0.2, and each update uses 10 episodes. In IPPO the actor and the value function of a cell only see that cell. In MAPPO the value function also sees the mean observation of the grid neighbours, which is the neighbourhood form of a centralised critic as in the DDPG critic, while the actor stays local. Both were trained for 8,000 episodes with the same checkpoint rule. Table 7.6 gives the outcome. All 12 runs end at almost the same reward,

Table 7.5: Training variants at 25 cells, held-out week reward

Variant	Seeds	Reward
Shared DDPG, 2,000 episodes, with forecast	3	0.881 ± 0.021
Expert-regularised loss (TD3+BC style), no forecast	3	0.844 ± 0.065
Neighbour communication in the actor	1	0.899
Expert-residual policy	1	0.894
Communication and expert residual	1	0.892
Long protocol, ensemble of three actors	3	0.904
Long protocol with extra observations, ensemble	2	0.892

between 0.907 and 0.908, whichever variant or seed is used. This is higher than the off-policy controller of the previous sections (0.904) and higher than the rich tuned rule (0.899): averaged over the runs the difference to the rich rule is $+0.009 \pm 0.002$, with a gain on 7 of 7 days, and every single run beats the rich rule on all seven days. The true-next-demand rule, which uses information no deployable controller has, is still higher at 0.920. IPPO and MAPPO give the same result, so in this environment the neighbourhood term in the value function brings no measurable benefit. The near-identical outcome of independent seeds suggests that the on-policy learner reliably finds the same good allocation behaviour, and that the remaining gap to the oracle comes from demand that cannot be predicted ten minutes ahead.

Table 7.6: On-policy baselines at 25 cells (8,000 episodes, checkpoint chosen on validation days), held-out week

Method	Seeds	Reward	Minus simple tuned expert	Minus rich tuned expert
IPPO	3	0.908 ± 0.000	+0.030	+0.009
MAPPO	9	0.908 ± 0.000	+0.030	+0.009
Shared DDPG, long protocol, ensemble	3	0.904	+0.026	+0.005

7.8 Robustness to Traffic Shifts

A controller trained on ordinary traffic may fail when the traffic changes. We therefore replayed the held-out week with the two scenario multipliers of Chapter 8, 6G growth and a massive-IoT surge. Table 7.7 gives the results for controllers that were not retrained.

When the traffic shifts, the learned controllers lose a lot of reward, and a rule that is re-tuned on the new traffic is clearly better. The forecast input helps the learned policy on ordinary traffic and under 6G growth (by about 0.03) and costs about 0.01 under the surge, so its effect is not consistent. Training on a mixture of scenarios solves the problem (Table 7.8). The policy trained on the mixture beats an expert re-tuned on the training days of each scenario in every

Table 7.7: Controllers trained on ordinary traffic, run on shifted traffic (25 cells, held-out week, DDPG-MARL averaged over three 2,000-episode seeds)

Traffic	**Equal split**	**Expert tuned on ordinary**	**Expert re-tuned on this traffic**	**MARL with forecast**	**MARL without forecast**
Ordinary traffic	0.750	0.877	0.877	0.881	0.851
6G growth (eMBB ×1.3, URLLC ×2, mMTC ×3)	0.615	0.762	0.762	0.724	0.692
Massive IoT (mMTC ×10)	0.666	0.725	0.735	0.672	0.685

traffic condition on average. The gain is clear on ordinary traffic and under 6G growth, and it is within noise under the massive-IoT surge.

Table 7.8: MARL trained on a mixture of traffic scenarios (two seeds, actors averaged), against an expert rule re-tuned on each scenario's own training days

Test traffic	**MARL reward**	**Difference (95% CI)**	**Days better**
Ordinary traffic	0.904	+0.026 ± 0.003	7/7
6G growth	0.780	+0.018 ± 0.002	7/7
Massive IoT	0.738	+0.003 ± 0.003	6/7

7.9 Earlier B5G MADDPG Development

Before the unified Milan environment existed, we implemented MADDPG on the B5G network-slicing release. The environment loads the graph, routing and slice data and makes one observation for every node that starts at least one flow. The policy is a shared actor with a critic that looks at the local state and allocation. The first comparisons used equal allocation, a rule-based heuristic and an untrained actor.

While developing it we found that the slice identifiers restart for each slice type. Using the identifier alone as the agent key therefore mixed up the records of eMBB, URLLC and mMTC and made the statistics invalid, so we excluded the affected results. The B5G release also has no usable latency and packet-loss fields, which an SLA-based reward needs. These experiments show that the replay, training and evaluation pipeline runs, but they do not show that the learned policy improves real QoS.

7.10 Chapter Conclusion

Three things came out of this chapter. First, a parameter-shared actor with a neighbourhood critic keeps its size and its reward when the network grows from 9 to 100 cells, while a central agent does not. Second, with a fair training budget, checkpoints chosen on validation days and averaged seeds, the learned controller beats a simply tuned expert rule on every test day at 9,

25 and 100 cells. Third, the off-policy controller only matches a richer rule that is tuned for each time of day, but the PPO-family learners, trained for 8,000 episodes, beat that rule on every test day in every run at 25 cells. Learned controllers also need retraining, or training on a mixture of scenarios, when traffic changes a lot. Whether the forecast helps the learned controller is discussed in Chapter 8.

Chapter 8

Integrated Evaluation and the Digital Twin

8.1 Introduction

This chapter evaluates the whole pipeline in the unified environment of Chapter 4. We ask whether knowing the demand of the next interval helps allocation, whether the forecast helps the learned controller, how the controllers behave over the whole city, and what a Digital Twin adds to the work. We then describe the Digital Twin dataset and its what-if scenarios, and finish by testing the surrogate environment against a packet-level simulator.

8.2 Value of the Forecast

On the held-out week the retrained forecaster of Chapter 6 has a mean squared error that is 18%, 33% and 31% lower than persistence for eMBB, URLLC and mMTC. To see what this is worth for allocation, we ran the same expert rule with three kinds of knowledge about the next ten minutes: the current demand, the TGNN forecast, and the true next demand, which is an upper bound for any forecast. Table 8.1 gives the result.

Table 8.1: Reward of the expert rule with different knowledge of the next ten minutes, held-out week (default targets)

Cells	**Equal split**	**Current demand**	**TGNN forecast**	**True next demand**	**Forecast minus current (95% CI)**	**Days better**	**Gap recovered**
9	0.785	0.794	0.825	0.945	+0.030 ± 0.008	7/7	20%
25	0.750	0.775	0.804	0.920	+0.029 ± 0.006	7/7	20%
49	0.728	0.758	0.788	0.902	+0.029 ± 0.004	7/7	20%
100	0.735	0.765	0.794	0.908	+0.029 ± 0.004	7/7	20%

caused by a bias of the block-trained forecaster in the outer cells, and the table above replaces it. Since this is a single day, it shows how the controllers behave and does not rank them.

8.5 The Digital Twin Dataset

No public dataset contains topology, per-slice traffic, QoS metrics, SLA outcomes and allocation decisions together, so we generated one from the unified environment (`src/unified/twin_dataset.py`). Each row is one cell in one 10-minute interval. The data is stored as Parquet files on the project drive, and summaries and samples are in the repository. We made two builds (Table 8.4), with 393.6 million rows in total.

Table 8.4: Size of the Digital Twin dataset

Build	**Cells**	**Days**	**Scenarios**	**Rows**	**Size**
Core year (central 20×20 block)	400	364	6	125,796,000	30.8 GB
Full city (whole grid)	10,000	62	3	267,810,000	40.5 GB
Total				**393,606,000**	

Provenance. The 62 recorded days are real Telecom Italia Milan data [34]. The other days of the core year are bootstrapped from real days of the same weekday, with 20% growth per year and a little random variation. The columns `source` and `source_date` say whether a row is real or bootstrapped. Latency and jitter follow curves calibrated on the 6G RAN telemetry. The radio fields follow the 3GPP CQI table [35] and the power follows the EARTH base-station model [36]. The slice mapping, the industrial zones, the per-user rates and the packet sizes are our own assumptions.

Structure. Each row has 122 columns. Twenty are for identity and calendar, 3 for control, 8 for events and state, 11 for radio and capacity, 6 for the neighbourhood and 3 for energy. There are 63 per-slice fields (21 for each slice, including the demand 10, 30 and 60 minutes ahead as forecasting targets), 2 for the reward and 6 for noisy telemetry. Appendix A lists all of them.

Validation. We ran all six scenarios for three days before the full build and checked five things. All scenarios have the same interval and cell rows. Traffic is conserved under energy saving, since the traffic that sleeping cells hand over equals what the neighbours receive. The industrial and massive-IoT multipliers are applied exactly. 1.0% of the telemetry is missing, as designed. The 10- and 60-minute targets equal the demand of the same cell one and six rows later. These are automated tests in the repository. They show that the generator is consistent with itself, not that it agrees with a packet-level simulator.

8.6 What-if Scenarios

All scenarios share the same days, the same behaviour policy and the same random noise, so any difference between two scenarios comes from the scenario. Table 8.5 gives the results for the core year.

Table 8.5: Scenarios of the core year

Scenario	**What changes**	**SLA met (eMBB / URLLC / mMTC)**	**Reward**	**Energy (MWh)**
baseline	Recorded traffic, no incidents	0.704 / 0.731 / 0.651	0.763	3,473
events	Adds flash crowds, outages, interference, URLLC bursts, mMTC storms, capacity degradation	0.703 / 0.730 / 0.651	0.763	3,473
growth_6g	eMBB ×1.3, URLLC ×2, mMTC ×3	0.553 / 0.696 / 0.610	0.688	3,648
energy_saving	Quiet cells sleep 01:00–05:00 and hand traffic to neighbours	0.703 / 0.737 / 0.659	0.766	3,435
industrial_urllc	Industrial cells: URLLC ×4 in weekday shifts	0.693 / 0.723 / 0.646	0.756	3,481
massive_iot	mMTC ×10, a further ×3 at the start of each hour	0.554 / 0.710 / 0.524	0.660	3,628

The scaled-traffic scenarios behave as intended. growth_6g lowers the eMBB compliance from 0.704 to 0.553, and massive_iot lowers the mMTC compliance from 0.651 to 0.524 and uses the most energy. The events scenario is almost the same as the baseline because over 99.8% of its rows have no event. The effect only shows when the rows are grouped by event type (Table 8.6).

Table 8.6: Effect of each injected event (core year, events scenario)

Event	**Rows**	**SLA met (eMBB / URLLC / mMTC)**	**Reward**
none	20,932,659	0.704 / 0.730 / 0.651	0.763
flash crowd	19,740	0.217 / 0.683 / 0.598	0.532
cell outage	985	0.000 / 0.095 / 0.019	0.019
interference spike	8,883	0.577 / 0.697 / 0.629	0.704
URLLC burst	814	0.608 / 0.510 / 0.575	0.631
mMTC storm	1,966	0.658 / 0.730 / 0.344	0.679
capacity degradation	953	0.323 / 0.710 / 0.647	0.578

Energy saving across the city. We also ran the energy-saving scenario on the whole city for 62 days (Table 8.7). When lightly loaded cells sleep from 01:00 to 05:00, night-time energy drops by 19.6%, from 1,983 MWh to 1,593 MWh, and total energy over the period drops by 3.2%. The price is that the URLLC compliance of the awake cells falls from 0.761 to 0.720, because their mean load rises from 0.045 to 0.066. The saving has to be read together with this loss of QoS.

Table 8.7: Night-time sleep across the city (01:00 to 05:00, 62 days)

Measure	**Baseline**	**Energy saving**
Night-time energy (MWh)	1983	1593 (19.6% lower)
Night cell-intervals asleep	0%	47.6%
SLA met, awake cells (eMBB / URLLC / mMTC)	0.983 / 0.761 / 0.724	0.976 / 0.720 / 0.693
Mean load of awake cells	0.045	0.066

8.7 The Digital Twin as an Approval Gate

A Digital Twin is only useful if it can check a decision before the decision is applied, and the unified environment now has a gate that does this. Before a cell's allocation is applied, the twin predicts the outcome of the next interval for the proposed allocation and for a safe fallback. It uses the QoS model of the environment and the TGNN forecast of the demand in that interval. The fallback is the expert rule tuned on the training days. The proposal is approved if its predicted reward is at least that of the fallback, and otherwise the fallback is applied. The gate works cell by cell.

We measured the gate on the held-out week with controllers whose proposals are sometimes bad, as when a learning agent explores online. These were the trained MARL actor with Gaussian noise of standard deviation σ added to its logits, and an equal split. Table 8.8 gives the reward with and without the gate.

Table 8.8: Effect of the approval gate on the held-out week (25 cells, mean over the three MARL seeds)

Controller	**Reward without gate**	**Reward with gate**	**Proposals approved**
MARL, logit noise $\sigma = 0$	0.900	0.898	72%
MARL, logit noise $\sigma = 0.5$	0.805	0.876	7%
MARL, logit noise $\sigma = 1.0$	0.707	0.877	3%
MARL, logit noise $\sigma = 2.0$	0.574	0.877	1%
Equal split	0.750	0.879	2%

The gate has two sides. For a good controller ($\sigma = 0$) it gives nothing: the reward is the same to within 0.003, and the gate still rejects about a quarter to a third of the proposals. For degraded controllers it recovers almost everything that was lost and brings each case back to the reward of the fallback alone (0.8775). So the gate is a safety measure and not an improvement. It stops the result from dropping below the fallback when a proposal is poor, and how well it works depends on how good the twin's prediction is. Our twin uses the same QoS model as the environment it is tested in, so this experiment cannot tell us how accurate the gate would be on a real network.

53%, and the load at which its 95th-percentile delay doubles is 68% rather than 93%. The surrogate is therefore more cautious than the simulator at moderate load and far too optimistic beyond saturation. The 5 ms floor in the sweep comes from the scheduling of the simulator. We read this as a check of shape and ranking, not as a calibration of absolute values.

Because the controllers were developed on the surrogate, we repeated the evaluation on a latency curve with the shape of the Simu5G sweep (flat, then a cliff at capacity). Table 8.10 gives the result. The ranking of the methods is the same as on the default curve: the forecast-fed rule beats the current-demand rule, and the learned controllers, which were not retrained, score about 0.95, above the rule tuned on this curve (0.942) and close to the rule that is given the true next demand (0.957). MAPPO retrained on this curve (three seeds) reaches 0.969, above that rule as well. The ordering of the controllers therefore does not depend on the smooth shape of the surrogate.

Table 8.10: Controllers on the Simu5G-shaped latency curve (25 cells, held-out week)

Controller	**Runs**	**Reward**
Equal split	1	0.814
Expert rule, untuned, current	1	0.909
Expert rule, tuned under this curve, current	1	0.913
Expert rule, untuned, forecast	1	0.940
Expert rule, tuned under this curve, forecast	1	0.942
Expert rule on the true next demand, untuned	1	0.957
IPPO, trained on the default curve, not retrained	3	0.953 ± 0.002
MAPPO, trained on the default curve, not retrained	9	0.953 ± 0.001
MAPPO, retrained under this curve	3	0.968 ± 0.000
DDPG-MARL, trained on the default curve, not retrained	3	0.948 ± 0.003

8.9 The Role of the Digital Twin in This Work

The Digital Twin has three roles in this work.

- **Training and evaluation.** The unified environment combines real Milan traffic with modelled slice demand, allocation actions, QoS estimates, SLA outcomes and rewards.
- **Scenario generation.** The 393.6-million-row dataset, six what-if scenarios and automated consistency checks allow controlled evaluation under different conditions.
- **Approval gate.** Section 8.7 shows that the twin can reject poor proposals before they are applied, which protects the result from a degraded controller.

The main limit is that the QoS comes from a calibrated surrogate whose reference telemetry is itself simulated. The fall of the mean gap from 0.086 to 0.021 shows a good fit, not an

independent validation, and the Simu5G sweep shows that the shape of the surrogate differs from a packet-level model. The twin therefore supports comparisons between controllers under assumptions that we state; it has not been validated for deployment decisions.

8.10 Summary

In the unified environment the TGNN forecast improves the expert rule by about 0.03 in reward at every network size and on all seven test days, which closes about one fifth of the gap to perfect knowledge. We have not shown that it improves the learned controller. Over the whole city the static split scores highest, because the rule starves the quiet slices, while in the dense block the predictive controller is best. The Digital Twin dataset supports six scenarios, among them an energy-saving policy that cuts night-time energy by 19.6% at a URLLC cost on the awake cells. The twin also works as an approval gate that protects the result from poor proposals, and a Simu5G sweep shows that the surrogate agrees with the simulator on the order of delays but not on the shape of the delay curve.

Chapter 9

Prototype: Slice Twin Monitor

9.1 Introduction

The prototype is *Slice Twin*, a monitor that runs in the browser and shows how different controllers split the bandwidth of each cell between eMBB, URLLC and mMTC over one day of real Milan traffic. It makes the environment of Chapter 4 and the experiments of Chapters 7 and 8 something you can click through. The source is in `app/slice_twin` of the project repository, and the monitor, a results dashboard and a project brief are published at `https://fyp-a09.github.io/6G/`.

9.2 Purpose and Design

Slice Twin lets you watch what the controllers do, see how incidents hit each slice, and compare forecast-based allocation with simpler allocation on the same data.

Data shown. One held-out day is replayed: Wednesday 6 November 2013, which the forecaster was not trained on (days 5 and 6 are excluded from its training for this reason). The traffic is real Telecom Italia Milan data [34]. We inject scripted incidents so that the controllers have something to react to: an mMTC storm, an interference spike, a cell outage, a backhaul degradation, a flash crowd and a URLLC burst on the central block. The city view adds four more, namely a firmware push in the north-west, wide-area interference in the east, storm damage in the south and a crowd at San Siro. Every incident is labelled and described in the interface. They are scripted for the demonstration and are not real events.

Interaction model. The interface is modelled on Cisco Packet Tracer. In Realtime mode the day plays continuously at a speed you choose. In Simulation mode the playback stops at each new event. An event list gives the time, cells and description of every incident, and a timeline marks the incidents and lets you jump to any time of the day.

9.3 Views and Controllers

Block view. The 100 central cells (10×10) are drawn as rings. The segments of a ring show how the cell splits its capacity between the three slices, and moving packets show the traffic that is served. Selecting a cell opens an inspector with its demand, its allocation, its latency against the target and the SLA status of each slice.

City view. All 10,000 cells are drawn as a map that can be coloured by real traffic, by load or by SLA misses, with landmarks (Duomo, Milano Centrale, San Siro, Linate airport, Navigli, Città Studi) and the active incidents. Selecting a cell compares the three controllers on it over the day, and a cell of the central block can be opened in the block view. City-wide curves show the bandwidth given to each slice and the fraction of cells meeting each SLA, with an option to show the gain over the Reactive controller.

Controllers. Three controllers run in the same environment (Chapter 4).

- **Static:** an equal split of each cell's capacity that never changes.
- **Reactive:** the expert rule, which gives URLLC and mMTC enough capacity to stay under their SLA-safe utilisation and the rest to eMBB, using the current demand.
- **Predictive:** the same rule fed with the TGNN forecast of the next ten minutes (the city-trained forecaster of Chapter 6 in the city view).

These are the controllers of Section 8.4, so the monitor agrees with Table 8.3. On the central block (Table 9.1) the Predictive controller is better than Reactive, and an oracle that knows the true next demand would be better still. Over the whole city the Static split scores highest.

Table 9.1: Controllers on the 100 central cells, Wednesday 6 November 2013

Controller	**SLA met (eMBB / URLLC / mMTC)**	**Reward**
Static	0.13 / 0.98 / 1.00	0.709
Reactive	0.59 / 0.74 / 0.57	0.735
Predictive	0.59 / 0.82 / 0.61	0.768
Oracle (true next demand)	0.59 / 1.00 / 1.00	0.876

9.4 Technical Notes

Technology and data flow. Slice Twin is a static web application with no code on a server. All the calculations are done beforehand, and the browser (HTML, CSS and JavaScript)

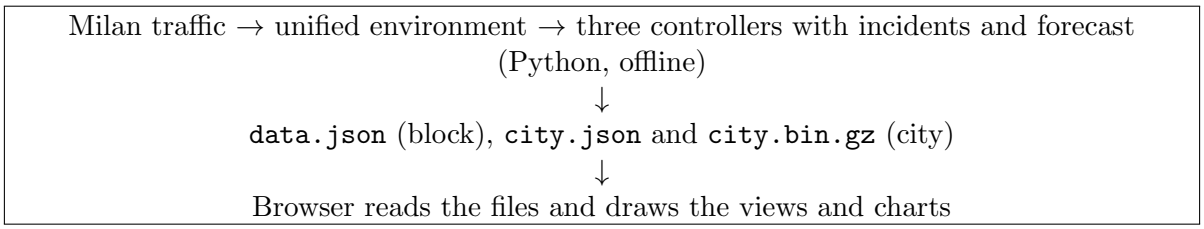


Figure 9.1: Data flow of the Slice Twin monitor

only reads the prepared files and draws the views (Figure 9.1). The files are produced by `showcase.py` for the block view and `showcase_city.py` for the city view, both in `src/unified`. The city data is stored compactly. For each interval and cell, one byte holds the log-scaled demand. For each controller, one byte holds the load (scaled by 250) and one byte holds flags: bits 0 to 2 say whether each slice meets its SLA, bit 3 marks an event and bit 4 an outage. The compressed binary file is a few megabytes, so the browser can handle 10,000 cells over 144 intervals and three controllers.

How to run it. From the root of the repository, `python -m http.server 8765 --directory app/slice_twin` starts the monitor at `http://localhost:8765`. To regenerate the data, run `showcase.py`, then `forecast_city_v2.py` once to train the city forecaster, and then `showcase_city.py`; the Milan files must be on the project drive. A GitHub Actions workflow publishes the monitor, the results dashboard and the project brief to GitHub Pages whenever the site or monitor folders change.

Demonstration. Figure 9.2 shows the city view at 17:30 on the demonstration day. In the top image the cells are coloured by real traffic, with the storm-damage incident in the south marked, and in the bottom image by SLA misses under the Reactive controller.

9.5 Contribution to the Project

The Slice Twin monitor is the working product of the project, and the code, the unified environment and the dataset in the repository stand behind it. It also works as a demonstration. A viewer can see that the predictive controller helps in the dense centre, that the static split holds up better on quiet outer cells, and how each incident affects each slice. Its limits are the limits of its data: it shows one held-out day with scripted incidents, and it does not control a live or simulated network. It is a tool for monitoring and comparison, not a deployed controller.

Chapter 10

Conclusion and Scope for further Research

10.1 Summary

This thesis asked two questions. Does a 6G slicing controller allocate bandwidth better when it knows the demand of the next interval? And does decentralised multi-agent reinforcement learning suit the task better than a single central agent? To find out, we built a pipeline of self-supervised learning, a temporal graph neural network and multi-agent reinforcement learning, and tested it in a unified slicing environment built on real Telecom Italia Milan traffic. All results come from a held-out week, and the checkpoints and rule targets were chosen on separate validation days.

Does knowing the next interval help? For the rule-based controller the answer is yes, by a modest amount. Replacing the current demand by the TGNN forecast raises the reward by about 0.03 at every size from 9 to 100 cells and on all seven test days, and it closes about one fifth of the gap to perfect knowledge (Chapter 8). The forecaster itself beats persistence on all 10,000 cells, with a mean squared error lower by 18%, 33% and 31% for eMBB, URLLC and mMTC. For the learned controller the answer is not settled: giving it the forecast changed its reward by less than the variation between seeds.

Is multi-agent learning better than a single central agent? Yes, in two ways. A shared actor of 19,332 parameters works from 9 to 100 cells, while the central agent needs 114,468 to 604,048 parameters and its reward falls from about 0.43 to 0.27 (Chapter 7). With a long training budget, checkpoints chosen on validation days and averaged seeds, the learned controller also beats a simply tuned expert rule on every test day at 9, 25 and 100 cells, by 0.023 to 0.026 in reward. The off-policy controller does not clearly beat a richer rule that is tuned for each time of day and cell class: its difference to that rule is between +0.001 and +0.005 and the interval includes zero at 25 and 100 cells. The on-policy learners (IPPO and MAPPO, 12 runs) do beat that rule: their mean reward is 0.908 against 0.899, a gain of 0.009

$\pm$ 0.002 on 7 of 7 days at 25 cells, and every run is better on all seven days. They do not reach the rule that is given the true next demand (0.920).

10.2 Contributions

1. **A unified slicing environment.** Since no public dataset joins topology, per-slice traffic, quality of service, SLA outcomes and allocation actions, we built an environment from real Milan traffic and grid data, with latency calibrated on public 6G RAN telemetry and load-coupled interference. Each component is marked as real, calibrated or modelled.

2. **A fair evaluation protocol for MARL.** Train, validation and test periods are separate, baselines are tuned on the training days, results are paired by day with intervals, and the strongest rule is part of the comparison.

3. **A measurement of what prediction is worth.** The value of the forecast is stated as the reward gained and as the fraction of the gap to perfect knowledge, at four network sizes.

4. **A city-scale forecasting finding.** A forecaster trained on the dense centre failed in the periphery, with error 10.8 times that of persistence, because it had learned the traffic level of the centre. Per-cell scaling, patch training and more training days removed the failure.

5. **A clean negative result for SSL.** Pretraining did not beat strong raw-feature baselines on de-duplicated 5G-NIDD flows at any label budget.

6. **A Digital Twin that checks decisions.** The approval gate recovers the reward lost to poor proposals, and a Simu5G sweep tests the surrogate against a packet-level simulator.

7. **A dataset and a prototype.** The dataset has 393.6 million rows and 122 columns, six what-if scenarios and injected incidents, and the Slice Twin monitor provides a visual interface to the results.

The work also records the corrections that changed our early results: a split in 5G-NIDD that followed the order of the labels, duplicate flows in 5G-NIDD, a slice-indexing error in the B5G environment, a city-wide bias in the forecast, and an earlier estimate of the value of the forecast that fell from 0.05 to 0.03 when we extended the test period to a week.

10.3 Limitations

Statistical strength. The test is one week (seven days, which are not independent) in one city, with at most three seeds per learned controller and one seed at 100 cells. The intervals we report describe the variation between those days, not between weeks, cities or training runs. A stronger study would use ten or more seeds and more weeks.

Comparison with the best hand-built rule. The off-policy learned controller only matches a rule tuned per time of day and cell class; the PPO-family learners beat it by about 0.009 reward, a small margin, and were tested only at 25 cells with the default QoS curve. The case for learning also rests on removing the manual tuning and on its behaviour as the network grows. The central-agent baseline is also weak: its reward fell during training, so the result shows that this central design performs poorly, not that every centralised design would.

Fidelity of the surrogate model. QoS comes from a calibrated surrogate, and the telemetry used for calibration is itself simulated. The latency spread was fitted on the same SLA-against-load curve that is used for evaluation, so the drop in the mean gap from 0.086 to 0.021 measures fit and is not independent validation. The Simu5G sweep agrees with the surrogate on the order of delays but not on their shape: the packet-level delay is flat until about 93% of capacity and then rises steeply, whereas the surrogate rises gradually from about half of capacity. The slice mapping, demand scale, interference coupling and reward weights are modelling assumptions.

Negative results. Self-supervised pretraining did not outperform raw features for 5G-NIDD classification or for the Milan forecast, so the contribution of the SSL stage to the pipeline is not demonstrated. The forecast has not been shown to help the learned controller. Over the whole city the static split beats the predictive controller because the rule starves quiet slices, and the city comparison covers a single day. The forecaster was trained once, without an ablation of its parts and without graph-free or published spatio-temporal baselines.

The Digital Twin claim. The twin trains controllers, evaluates them, generates scenarios and gates proposals. The gate uses the same QoS model as the environment in which it is tested, so how well it would predict outcomes on a real network is not shown. The controllers have not been validated against a live or packet-level network. The prototype is a monitor of one held-out day with scripted incidents.

10.4 Future Work

Stronger statistics and baselines. Repeat the experiments with ten or more seeds and several test weeks, add the 100-cell runs with more seeds, and add a graph-free forecaster and published spatio-temporal models together with an ablation of the city forecaster.

Closing the simulator gap. Connect the project's own 19-gNodeB scenario to Simu5G, run controllers in closed loop with it and record latency, jitter and loss, which would also supply the latency and loss data that the B5G dataset lacks. Calibrate the surrogate on the measured delay curve, in particular the flat-then-cliff shape.

SSL with richer inputs. Test SSL on inputs where hand-crafted features are not available, such as raw radio measurements or longer sequences, and with an encoder that uses neighbouring cells and earlier intervals.

Forecast-aware learning. Investigate why the forecast does not help the learned controller, for example with a longer training budget, a design in which the forecast is part of the critic, or training on forecast errors.

Semantic slicing and O-RAN traces. Extend the reward with semantic criteria suggested by the literature, and check the surrogate against public O-RAN and testbed traces that contain measured PRB use, throughput and channel quality.

Deployment. Move the monitor and controllers from replay towards a closed loop with a simulator or testbed, and study decision latency inside the control interval, coordination between neighbouring cells and the cost of reconfiguration.

10.5 Concluding Remarks

Within these limits, the work shows three things. Predicting traffic before allocating bandwidth helps a rule-based controller. A parameter-shared multi-agent controller scales to larger networks where a central one does not, and it beats rules whose targets were tuned on the training days. And a forecaster has to be trained on the whole city if it is going to be used there. Two questions stay open: whether a learned controller can make real use of the forecast, and whether a packet-level simulator agrees with our surrogate beyond the ordering of delays.

Appendix A

Dataset Columns and Experimental Settings

A.1 Columns of the Digital Twin Dataset

Each row of the dataset is one cell in one 10-minute interval and has 122 columns. 59 are general columns and 21 are repeated for each of the three slices with the prefix `eMBB_`, `URLLC_` or `mMTC_`, giving 63. The general columns are:

scenario	policy	modulation
interval	idle_share	spectral_efficiency_bps_hz
day_index	event_type_code	power_w
date	event_type	energy_wh
interval_of_day	event_id	energy_efficiency_mbit_per_j
hour	anomaly	rehomed_in_mbps
minute	cell_asleep	rehomed_out_mbps
weekday	capacity_factor	handover_users_est
month	interference_factor	reconfiguration_churn
day_of_year	capacity_mbps	anomaly_score
iso_week	spare_capacity_mbps	all_slices_sla_ok
season	total_demand_mbps	reward
is_weekend	total_served_mbps	team_reward
is_holiday	prbs_used	obs_total_demand_mbps
is_business_hours	load	obs_embb_latency_p95_ms
is_night	neighbour_load	obs_load
source	neighbour_mean_demand_mbps	obs_sinr_db
source_date	neighbour_all_sla_ok	obs_cqi
cell_id	sinr_db	telemetry_missing
zone	cqi	

The per-slice columns (to be prefixed with the slice name) are:

demand_mbps	share	queue_kbit
demand_next_10min_mbps	prbs	latency_p50_ms
demand_next_30min_mbps	allocated_mbps	latency_p95_ms
demand_next_60min_mbps	utilisation	jitter_ms
demand_1h_mean_mbps	served_mbps	loss
demand_delta_mbps	throughput_per_user_mbps	sla_margin_ms
users_est	packets	sla_ok

A.2 Experimental Settings

- **Data split.** Training days 1 to 40; validation days 41 to 44; test days 45 to 51 (16–22 December 2013). Days 5 and 6 are excluded from forecaster training.
- **Environment.** 10-minute intervals, 144 per day, capacity 100 Mbps per cell, interference coupling 0.3, reward $0.7\,S + 0.3\,F - 0.1\,H$, headroom targets $(0.65, 0.5, 0.65)$ by default.
- **Forecaster.** Two GraphSAGE layers and a GRU cell with 64 hidden units, residual output head, window of six intervals, 32×32 patches, per-cell 95th-percentile scaling, trained on 39 days with the epoch chosen on days 41 to 44.
- **MARL.** Shared actor of 19,332 parameters, neighbourhood critic, discount 0.9, $\tau = 0.01$, learning rates 3×10^{-4} and 10^{-3}, replay 50,000; long protocol 4,000 episodes with validation every 250 episodes, seeds 0 to 2.
- **PPO family.** 8,000 episodes, ten episodes per update, clip 0.2, four epochs, entropy bonus 0.003, GAE with $\lambda = 0.95$.
- **Code and results.** Everything is in the repository `FYP-A09/6G`: environment and learners in `src/unified`, result files in `reports/unified`, figures in `docs/figures`, tests in `tests/test_unified.py`.

References

[1] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] Q. Li, Y. Zhang, L. Wang, X. Duan, *et al.*, "Native network digital twin architecture for 6G: From design to practice," *IEEE Communications Magazine*, vol. 63, no. 4, pp. 24–31, 2025. Online publication 2024; issue publication 2025. Confirm full author list from IEEE Xplore.

[3] "A graph neural network-based digital twin for network slicing management," 2022. Bibliographic record not independently verified. Do not cite until the exact paper, authors, and venue are confirmed.

[4] A. Calvillo-Fernandez, T. Dimitrovski, M. Groshev, A. Ganesh, C. Ayimba, and A. de la Oliva, "Attention to virtualization: Making network digital twins aware of network slicing," *IEEE Network*, vol. 39, no. 3, pp. 134–139, 2025.

[5] M. Yaqoob, R. Trestian, M. Tatipamula, and H. X. Nguyen, "Digital-twin-driven end-to-end network slicing toward 6G," *IEEE Internet Computing*, vol. 28, no. 2, pp. 47–55, 2024. First published online in 2023; issue publication is 2024.

[6] A. Ali and H. Arslan, "AI-driven digital twins: Optimizing 5G/6G network slicing with NTNs," 2025. Preprint record; verify any claimed journal publication separately.

[7] J. Sengendo and F. Granelli, "Toward self-optimizing 6G networks through network digital twin intelligence: A real network traffic evaluation," *IEEE Networking Letters*, vol. 8, pp. 174–178, 2026.

[8] "Revolutionizing wireless networks with self-supervised learning: A pathway to intelligent communications," 2024. Exact title/authors/venue not independently verified. Confirm the original source before citing.

[9] "Network-to-network: Self-supervised network representation learning via position prediction," 2025. Exact bibliographic record not independently verified. Search result/title may differ from the published paper title.

[10] "Encrypted network traffic classification in SDN using self-supervised learning," 2022. Exact bibliographic record not independently verified. Confirm the title and venue from the original paper.

[11] E. Eslami and W. Hamouda, "Network traffic classification using self-supervised learning and confident learning," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 9100–9120, 2025.

[12] J. Zhou, W. Zhao, S. Chen, *et al.*, "Dynamic network slice scaling assisted by prediction in 5G network," *IEEE Access*, 2020. Confirm final volume, pages, and complete author list from IEEE Xplore.

[13] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Perez, "DeepCog: Optimizing resource provisioning in network slicing with AI-based capacity forecasting," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 2, pp. 361–376, 2020.

[14] "Self-supervised spatiotemporal graph neural networks with self-distillation for traffic prediction," 2023. Exact title/authors/venue not independently verified. Confirm the original publication before citing.

[15] Y. Li, H. Wang, G. Min, X. Wang, *et al.*, "A flexible and scalable multi-agent learning framework for dynamic RAN slicing in 6G native-AI networks," *IEEE Transactions on Mobile Computing*, 2026. DOI indicates online-first record in 2025; verify issue, volume, pages, and full author list.

[16] "Multi-agent reinforcement learning for slicing resource allocation in vehicular networks," 2023. Exact title and claimed IEEE Transactions on Intelligent Transportation Systems venue not independently verified. Confirm before citing.

[17] G. Zhou, L. Zhao, G. Zheng, Z. Xie, S. Song, and K.-C. Chen, "Joint multi-objective optimization for radio access network slicing using multi-agent deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 9, pp. 11828–11843, 2023.

[18] D. B. John, "GAN-enhanced deep reinforcement learning for semantic-aware resource allocation in 6G network slicing," 2026. The exact arXiv record and author/title combination were not independently verified; do not assume this is a published paper.

[19] Q. Liu, N. Choi, and T. Han, "Deep reinforcement learning for end-to-end network slicing: Challenges and solutions," *IEEE Network*, vol. 37, no. 2, pp. 222–228, 2023. First published online in 2022.

[20] M. Bagaa, D. L. C. Dutra, T. Taleb, and H. Flinck, "Toward enabling network slice mobility to support 6G system," *IEEE Transactions on Wireless Communications*, vol. 21, no. 12, pp. 10130–10144, 2022.

[21] M. Helmy, A. A. Abdellatif, N. Mhaisen, A. Mohamed, and A. Erbad, "Slicing for AI: An online learning framework for network slicing supporting AI services," *IEEE Transactions on Network and Service Management*, vol. 22, no. 6, pp. 5239–5254, 2025.

[22] "An agentic AI control plane for 6G network slice orchestration, monitoring, and trading," 2026. Exact bibliographic record not independently verified. Confirm authors, publication venue, and stable source before citing.

[23] M. R. Chowdhury, E. Hammad, L. Loven, S. Pirttikangas, A. P. da Silva, and W. Saad, "A framework for AI-native semantic-based dynamic slicing for 6G networks," 2025.

[24] J. Yoon, Y. Zhang, J. Jordon, and M. van der Schaar, "VIME: Extending the success of self- and semi-supervised learning to tabular domain," in *Advances in Neural Information Processing Systems*, 2020.

[25] D. Bahri, H. Jiang, Y. Tay, and D. Metzler, "SCARF: Self-supervised contrastive learning using random feature corruption," in *International Conference on Learning Representations*, 2022.

[26] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[27] I. Rubachev, A. Alekberov, Y. Gorishniy, and A. Babenko, "Revisiting pretraining objectives for tabular deep learning." arXiv:2207.03208, 2022.

[28] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[29] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," in *Advances in Neural Information Processing Systems*, 2021.

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms." arXiv:1707.06347, 2017.

[31] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2022.

[32] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. S. Torr, M. Sun, and S. Whiteson, "Is independent learning all you need in the StarCraft multi-agent challenge?." arXiv:2011.09533, 2020.

[33] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations*, 2016.

[34] G. Barlacchi *et al.*, "A multi-source dataset of urban life in the city of Milan and the province of Trentino," *Scientific Data*, vol. 2, p. 150055, 2015.

[35] 3rd Generation Partnership Project (3GPP), *NR; Physical Layer Procedures for Data*, 2024. Use the exact release/version cited in the implementation or thesis.

[36] G. Auer, V. Giannini, C. Desset, I. Godor, P. Skillermark, M. Olsson, M. A. Imran, M. J. Sabino, M. J. Gonzalez, G. De La Roche, *et al.*, "How much energy is needed to run a wireless network?," *IEEE Wireless Communications*, vol. 18, no. 5, pp. 40–49, 2011.

List of Publications

(Based on the Research Work)

Papers Published

None so far.

Papers Communicated

None so far.

Planned Publications

Survey Paper (in preparation)

1. **Title:** Digital Twin-Assisted Network Slicing for 6G Using Self-Supervised Learning, Temporal Graph Neural Networks and Multi-Agent Reinforcement Learning: A Survey

2. **Journal:** IEEE Communications Surveys & Tutorials (tentative)

Technical Paper (in preparation)

1. **Title:** Forecast-Informed Multi-Agent Control of 6G Network Slices in a City-Scale Digital Twin

2. **Journal:** IEEE Transactions on Network and Service Management (tentative)